Technical Report: Algorithmic Engine for Hierarchical VHDL Gate Levelization using Python

Sharvari Patwardhan
Roll Number: **26M1145**
Department of Electronic Systems (EE5)
M.Tech TA

August 31, 2026

Abstract

This report presents a Python program designed to analyze VHDL digital circuits and calculate gate logic levels. The tool reads VHDL netlist files, parses input and output connections, and breaks down complex hierarchical sub-modules into basic primitive logic gates. It then builds a dependency graph of all connected components to compute the exact depth of each gate and determine the critical path of the circuit.

Contents

1 Introduction

This project takes a VHDL design as input, identifies all the individual logic gates, and calculates the depth level of each gate from the inputs to the outputs.

2 Algorithm & System Architecture

The program works in four simple steps to process the VHDL files:

1. **VHDL Parsing:** Reads the VHDL files to find all inputs, outputs, sub-modules, and connections.
2. **Module Cataloging:** Creates a dictionary of all sub-modules so the system knows their input and output ports.
3. **Hierarchy Flattening:** Breaks down all complex sub-modules into basic gates (like `AND`, `OR`, `NAND`, and `INVERTER`).
4. **Levelization:** Connects the gates into a graph and calculates the level (depth) of each gate starting from the primary inputs.

2.1 Calculating Gate Levels

The level $L(g)$ of any gate g is calculated as:

$$L(g) = \begin{cases} 1 & \text{if } g \text{ is connected directly to primary inputs} \\ \max(\text{levels of previous connected gates}) + 1 & \text{otherwise} \end{cases}$$

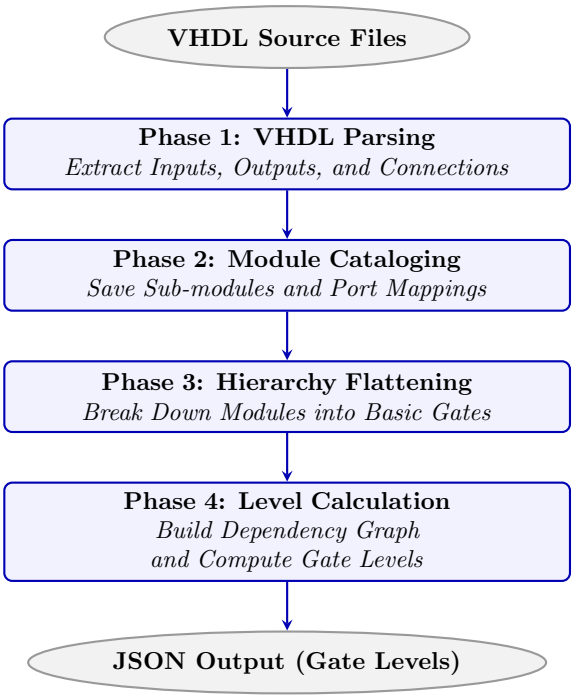

Figure 1: Engine Execution Flow for VHDL Parsing, Flattening, and Level Calculation

3 Software Design & Implementation Details

The program uses Python's `re` library to extract VHDL syntax constructs and the `networkx` library to perform topological sorting for level calculation[cite: 2].

3.1 VHDL Parsing Engine

The parser extracts primary inputs, primary outputs, and individual gate component instantiations using regular expressions.

```
instances = re.findall(
    r"(\w+)\s*:\s*(\w+)\s+port\s+map\s*\((.*?)\);",
    content,
    re.DOTALL | re.IGNORECASE
)
```

Listing 1: Extracting Component Instantiations in VHDL

3.2 Package Parsing & Signal Driver Mapping

When a VHDL package file is supplied, the tool parses component definitions to resolve input and output directions[cite: 2]. Output wires are mapped directly to their driving gate instances[cite: 2]:

```
# Extract port directions from package definitions
for comp_name, port_block in components:
    comp_name = comp_name.strip().upper()
    component_defs[comp_name] = {"in": [], "out": []}

# Map output signals to their driving gate instance
for inst_name, gate_type, map_block in instances:
    # Resolve input and output wires based on named/positional mapping
    gate_input_signals[inst_name] = in_wires
    for out_wire in out_wires:
        signal_driver_gate[out_wire] = inst_name
```

Listing 2: Mapping Signal Drivers to Logic Gates

3.3 Graph Construction & Level Calculation

A directed acyclic graph (DAG) is constructed using `networkx`[cite: 2]. Nodes represent individual gate instances, and directed edges represent wire signal dependencies[cite: 2]:

```
G = nx.DiGraph()

# Add direct gate-to-gate dependencies
for target_gate, driving_gates in gate_dependencies.items():
    G.add_node(target_gate)
    for driver in driving_gates:
        G.add_edge(driver, target_gate)

# Calculate levels via topological sort
topological_order = list(nx.topological_sort(G))
for gate in topological_order:
    predecessors = list(G.predecessors(gate))
    if not predecessors:
        gate_levels[gate] = 1
    else:
```

```
16          gate_levels[gate] = max(gate_levels[p] for p in predecessors) +
        1
```

Listing 3: Building Graph and Computing Levels

3.4 Execution & Usage Guide

The script supports execution for both standalone netlists and designs requiring external package definitions[cite: 2].

3.4.1 Prerequisites

Install the required dependency[cite: 2]:

```
1 pip install networkx
```

3.4.2 Command-Line Interface

- **Standalone Execution:**

```
1 python levelizer.py -i "path/to/circuit.vhd"
```

- **Execution with Custom Package Library:**

```
1 python levelizer.py -i "path/to/circuit.vhd" -p "path/to/Gates.vhdl
      "
```

- **Exporting Results to JSON:**

```
1 python levelizer.py -i "path/to/circuit.vhd" -p "path/to/Gates.vhdl
      " -o "path/to/output.json"
```

4 Experimental Validation

4.1 Test Case: 5-Bit Fibonacci Detector

To test the program, a 5-bit Fibonacci detector circuit was used. The circuit checks if a 5-bit input number is a Fibonacci number $(0, 1, 2, 3, 5, 8, 13, 21)$ using 17 logic gates (`g1` through `g17`).

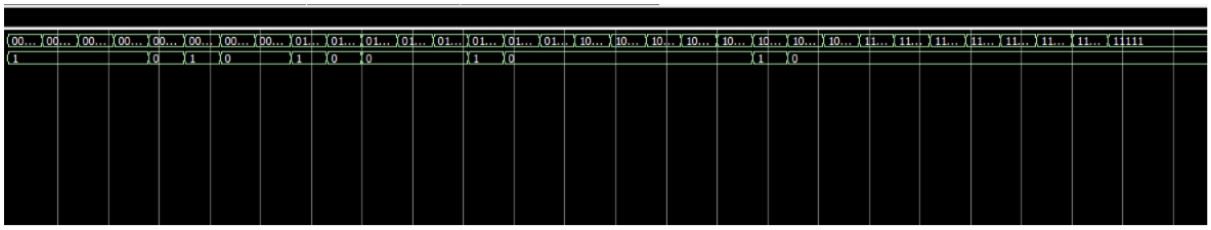

Figure 2: Quartus II Simulation Waveform for 5-Bit Fibonacci Detector

4.2 Circuit Netlist Input

The circuit netlist diagram processed by the Python engine:

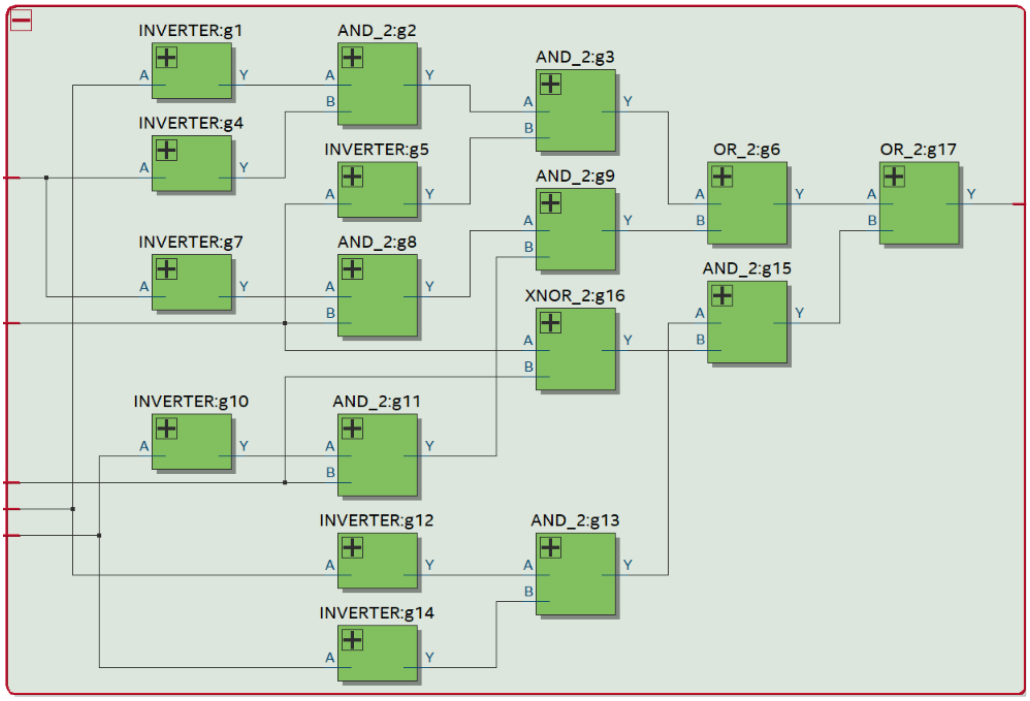

Figure 3: Quartus Netlist Diagram for 5-Bit Fibonacci Detector

4.3 Output Results

Running the Python script produces the calculated level for each gate:

```
--- LEVELIZATION RESULTS ---
{
    "primary_inputs": [
        "x0",
        "x1",
        "x2",
        "x3",
        "x4"
    ],
    "primary_outputs": [
        "output"
    ],
    "gate_node_levels": {
        "g1": 1,
        "g4": 1,
        "g5": 1,
        "g7": 1,
        "g10": 1,
        "g12": 1,
        "g14": 1,
        "g16": 1,
        "g2": 2,
        "g8": 2,
        "g11": 2,
        "g13": 2,
        "g3": 3,
```

```
27          "g9": 3,
28          "g15": 3,
29          "g6": 4,
30          "g17": 5
31      }
32  }
```

Listing 4: Calculated Gate Levels Output

The engine calculated a total maximum depth of **5 levels**, ending at gate `g17`.

5 Conclusion

The Python tool successfully reads VHDL netlists, breaks down complex sub-modules into basic gates, and calculates gate levels. By organizing the gates into a graph, the tool easily finds the total depth and longest path through the circuit.